



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,

Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560078



DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING DSCE, BENGALURU

**Alternate Assessment Tool Report
Submitted for the Course**

SOFTWARE ENGINEERING & SOFTWARE TESTING – IPCC22IS63

**On the Topic
“SECURITY AND PRIVACY ANALYSIS OF META VIRTUAL
REALITY APPLICATIONS”**

Submitted by

Bhoomiks Nayak (1DS23IS031)

Chandra Shekar GV (1DS23IS035)

Chetan S (1DS23IS037)

Dikshith M (1DS23IS044)

(Group No.: A08)

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATKA
2025-26**

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING



DECLARATION

We declare that we abide by the ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. The work submitted in this report of **Software Engineering & Software Testing (IPCC22IS63)**, *VI Semester BE, ISE* has been compiled by referring to the relevant online and offline resources to the best of our understanding and in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Information Science & Engineering, at Dayananda Sagar College of Engineering, an autonomous institution affiliated to VTU, Belagavi during the academic year 2025-26.

We hereby declare that the same has not been submitted in part or full for other academic purposes.

Bhoomiks Nayak (1DS23IS031)

Chandra Shekar GV (1DS23IS035)

Chetan S (1DS23IS037)

Dikshith M (1DS23IS044)

Place:

Date:

ABSTRACT

Virtual Reality (VR) has emerged as a transformative technology driving the development of immersive metaverse applications. However, the rapid proliferation of VR apps brings critical security and privacy challenges that have largely gone unexamined. This paper presents VR-SP Detector, a comprehensive security and privacy assessment tool designed specifically for Meta VR applications. The tool integrates program static analysis, privacy-policy analysis, and user review analysis to holistically evaluate vulnerabilities in VR applications. Using the VR-SP Detector, the study conducts an empirical analysis of 900 popular VR apps sourced from the SideQuest app store and installed on the Meta Quest 2 device. The analysis encompasses manifest vulnerability profiling, OS-related and VR-platform-related security assessments, PII data leak identification, privacy policy consistency checks, and user review analysis. Key findings reveal that over 95.40% of apps lack root detection, 37.20% use insecure random generators, and 44.40% invoke biometric data collection functions without corresponding permission declarations. Furthermore, 55.20% of apps provide no privacy policy, and 11.00% of those that do contain contradictory statements. User reviews corroborate these concerns, with 58.63% of security-related reviews specifically mentioning biometric data. Based on these results, the study offers eight targeted development recommendations to improve the security, privacy, and policy compliance of future VR applications.

Keywords: *Virtual Reality, Metaverse, Security and Privacy, Static Analysis, VR-SP Detector, Biometric Data, Privacy Policy, PII Data Leaks, Android, Unity.*

CONTENTS

SL NO.	TITLE	PAGE NO.
1.	INTRODUCTION	5
2.	FUNCTIONAL REQUIREMENTS	6
3.	NON-FUNCTIONAL REQUIREMENTS	7
4.	OVERVIEW	7
5.	SOFTWARE ENGINEERING PRINCIPLES REFLECTED IN THE WORK	9
6.	SE/AI TOOLS USED	11
7.	CRITICAL ANALYSIS OF SE PRACTICES	12
8.	REAL LIFE CASE STUDY	13
9.	MAPPING SE PRINCIPLES	15
10.	BENEFITS OF APPLYING SE PRINCIPLES TO THE SYSTEM	16
11.	CONCLUSION	17
12.	REFERENCES	18
13.	ORIGINAL RESEARCH PAPER	19

1. INTRODUCTION

Virtual Reality (VR) represents one of the most rapidly evolving frontiers in computing, driven by a growing ecosystem of immersive metaverse applications. Industry projections estimate the global VR market will expand from \$25.11 billion in 2023 to \$165.91 billion by 2030, reflecting the accelerating pace of adoption across gaming, education, social interaction, and commerce. Companies such as Meta, Apple, Sony, HTC, and ByteDance have invested heavily in VR hardware and software platforms, resulting in a rich and diverse landscape of VR applications.

Despite this rapid growth, VR applications carry significant security and privacy risks that have not received proportionate research attention. Most VR operating systems are built upon established mobile platforms such as Android, meaning that VR apps inherit the well-documented security vulnerabilities of mobile applications. These include insecure flag settings in manifest files, dangerous permission misuse, PII data leaks, and the use of vulnerable third-party libraries (TPLs). However, VR apps present a more complex threat surface beyond conventional mobile vulnerabilities. They rely on specialized 3D game engines such as Unity and Unreal Engine, and they extensively collect human biometric data—including hand-tracking, eye-tracking, face-tracking, and body-tracking information—to deliver immersive experiences. The collection of such highly sensitive data, often without proper permission declarations or transparent privacy policies, creates profound privacy risks for users.

A particularly troubling finding from existing literature is the case of Bigscreen, a popular virtual social VR app in which a vulnerability allowed unauthorized parties to activate a user's microphone and eavesdrop on private conversations without consent. Such incidents underscore the urgency of systematic security and privacy evaluation of VR applications.

To address this gap, the research paper introduces VR-SP Detector, a comprehensive automated security and privacy assessment tool tailored for Meta VR applications. It was applied to 900 popular VR apps from the SideQuest app store—more than five times the scale of prior comparable studies—yielding a rich set of empirical findings and concrete development recommendations.

This report analyzes the VR-SP Detector study through the lens of Software Engineering principles, examining its functional and non-functional requirements, architectural design, and alignment with established SE practices. A real-world case study and a mapping of SE principles further contextualize the practical relevance of the research.

2. FUNCTIONAL REQUIREMENTS

The VR-SP Detector system, as described in the IEEE paper, fulfills the following functional requirements:

- Collect and process VR application APK files from both the official Meta VR store and the SideQuest third-party store.
- Perform manifest analysis of each APK to extract app basic information, permission usage, activity launch modes, and security-related flags such as allowBackup, debuggable, and clearTextTraffic.
- Detect OS-related security and privacy vulnerabilities through static analysis of decompiled Java and Smali files, covering nine vulnerability categories including SQL Database Injection, Insecure Certificate Validation, Insecure Random Generator, and Root Detection absence.
- Identify third-party library (TPL) usage and detect the presence of known vulnerable TPLs, as well as dangerous permission requests within TPL packages.
- Detect VR-platform-specific vulnerabilities including Unity IAP (In-App Purchasing) vulnerabilities and inconsistent or unauthorized use of human biometric data collection functions (hand-tracking, eye-tracking, face-tracking, body-tracking).
- Perform PII data leak identification using taint analysis, tracking the propagation of sensitive data from defined sources to potentially unsafe sinks.
- Analyze privacy policies of VR apps for contradictory statements, GDPR compliance violations, and inconsistencies between stated policies and actual code behavior.
- Conduct user review analysis by keyword-based classification of SideQuest reviews, identifying user concerns related to sensitive data and biometric permissions.
- Generate comprehensive security and privacy assessment reports with actionable findings for each analyzed VR application.

3. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements governing the design and operation of the VR-SP Detector are as follows:

- **Performance:** The tool must efficiently process large volumes of APK files. In this study, it analyzed 900 VR applications, necessitating optimized static analysis pipelines and batch processing capabilities.
- **Scalability:** The system is designed to scale beyond the 900-app dataset, as evidenced by its extensible architecture and use of general-purpose analysis tools such as Androguard and LibScan.
- **Accuracy:** The taint analysis and pattern-matching components must minimize false positives. The study manually validated 20% of identified paths and reported no false positives in sampled outputs.
- **Reliability:** Privacy policy analysis components such as PolicyLint and GDPRWise must produce consistent and reproducible results across different app categories.
- **Maintainability:** The modular design of VR-SP Detector, separating app analysis, PII leak identification, privacy policy analysis, and review analysis into independent modules, ensures that individual components can be updated or replaced without affecting others.
- **Generalizability:** The tool is designed to be applicable across different VR app categories—virtual society, gaming, art and culture, education, and business—rather than being restricted to a single domain.
- **Transparency:** All findings, datasets, and tool implementations are made publicly available at the project GitHub repository to support reproducibility and further research.

4. OVERVIEW

The paper presents the VR-SP Detector, an end-to-end automated framework for evaluating the security and privacy posture of Meta VR applications. The system operates across five integrated modules as illustrated in its overall architecture: App Collection, VR App Analysis, PII Data Leaks Identification, Privacy Policy Analysis, and Review Analysis.

App Collection: A total of 900 popular VR apps were collected from the SideQuest app store and the official Meta VR store. Apps were installed on a Meta Quest 2 device, and APK files were extracted using the SideQuest APK backup function. The dataset spans five categories: 109 virtual social apps, 503 gaming apps, 128 art and culture apps, 120 education apps, and 40 business and finance apps.

VR App Analysis: Each APK was decompiled using Androguard to extract the AndroidManifest.xml configuration file, Java bytecodes (classes.dex), and Smali intermediate representations. Three sub-analyses were performed: (1) Manifest Analysis to extract app metadata, permission declarations, and activity launch modes; (2) OS-related vulnerability detection using pre-defined pattern matching across nine vulnerability types; and (3) VR-platform-related vulnerability detection focusing on Unity IAP vulnerabilities and biometric data collection inconsistencies.

PII Data Leaks Identification: A taint analysis framework was applied to trace the propagation of sensitive PII data from defined taint sources—such as `getString()`, `getIntent()`, and `getAddress()`—to potentially unsafe taint sinks—such as `sendBroadcast()` and `sendDataMessage()`.

Privacy Policy Analysis: Privacy policies were analyzed using PolicyLint to extract structured statements in the format `<entity, action, data type>`. Contradictory statements were identified for apps that both claimed to collect and not collect the same data type. GDPR compliance checks were performed using GDPRWise. Consistency checks cross-referenced permission declarations, biometric function usage from decompiled code, and privacy policy statements.

Review Analysis: A total of 7,772 user reviews from 897 apps were collected from SideQuest. Reviews were filtered for English language content and analyzed using keyword matching across predefined categories including biometric, location, media, network, and general permission-related terms.

The empirical study yielded seven key findings across the five research questions, revealing widespread security vulnerabilities, pervasive privacy policy non-compliance, and significant inconsistencies between declared permissions and actual biometric data collection behaviors in VR applications.

5. SOFTWARE ENGINEERING PRINCIPLES REFLECTED IN THE WORK

a. Modularity

The VR-SP Detector is architecturally organized into five distinct, independently operable modules: App Collection, VR App Analysis, PII Data Leaks Identification, Privacy Policy Analysis, and Review Analysis. Each module addresses a specific dimension of security and privacy assessment without coupling to the internal logic of others. This modularity allows individual components—such as the privacy policy checker or the taint analysis engine—to be updated, replaced, or extended independently as the VR ecosystem evolves.

b. Abstraction

Complex low-level operations are abstracted behind well-defined interfaces and tool integrations. For instance, the intricacies of APK decompilation are abstracted through the Androguard library, while privacy policy parsing complexities are abstracted through PolicyLint. The security analysis of Unity-developed apps abstracts binary analysis through static native binary analysis frameworks, enabling higher-level reasoning about payment and biometric data vulnerabilities without direct manipulation of compiled binaries.

c. Maintainability

The use of established open-source analysis tools such as Androguard, LibScan, PolicyLint, and GDPRWise ensures that the VR-SP Detector can be maintained and updated as these underlying tools evolve. The pre-defined vulnerability pattern rules (Table II in the paper) are structured in a manner that allows new patterns to be appended as novel vulnerability classes emerge in the VR ecosystem, minimizing rework.

d. Reusability

The pattern-matching rules for OS-related vulnerabilities are directly adapted from prior work on COVID-19 contact tracing app analysis, demonstrating the reuse of established analysis methodologies in a new domain. Similarly, the taint analysis framework for PII data leaks is adapted from FlowDroid and extended to cover VR-specific PII types such as VR device IDs and controller IDs.

e. Testing and Validation

The system incorporates rigorous validation at multiple levels. Taint analysis results were manually

verified against 20% of identified propagation paths, yielding no false positives in the sampled set. GDPR compliance results were validated against 50 manually reviewed privacy policies. The precision of the pre-defined vulnerability detection patterns is grounded in prior work claiming 96.19% precision, further validated by the manual checks conducted in this study.

f. Scalability

The VR-SP Detector demonstrates strong scalability by operating on a dataset of 900 apps—approximately five times the scale of the closest prior study (OVRSEEN). The use of automated, pipeline-based analysis tools ensures that the system can be extended to larger datasets as the VR app ecosystem continues to expand, without requiring manual intervention for each additional application.

g. Documentation and Benchmarking

The study contributes a publicly available dataset of 900 VR app analyses, along with the full VR-SP Detector tool, at <https://github.com/Henrykwokkk/Meta-detector-expand>. This represents a significant benchmarking contribution, providing the research community with a standardized and reproducible basis for future security and privacy analyses of VR applications.

h. Iterative Development and Lifecycle Practices

The study explicitly builds upon a preliminary study that analyzed 500 VR apps, extending the dataset to 900 apps and adding two new modules: TPL analysis and user review analysis. This iterative extension reflects sound software engineering lifecycle practices, where each iteration refines and expands the system based on identified gaps from prior versions.

6. SE AND AI TOOLS USED

Software Engineering Tools

Androguard: An open-source Python-based APK analysis tool used to decompile VR app APK files, extract AndroidManifest.xml configuration files, parse classes.dex bytecodes, and construct method call graphs for vulnerability pattern matching.

LibScan: A third-party library detection tool employing method-opcode similarity and call-chain-opcode similarity to identify TPL usage in VR apps. Used to detect 205 known vulnerable TPLs and 255 general TPLs across the app dataset.

PolicyLint: An NLP-based privacy policy analysis tool that parses privacy policy text into structured statements of the form <entity, action, data type>, enabling automated detection of contradictory statements within app privacy policies.

GDPRWise: A free online service used to perform GDPR compliance checks on app privacy policies, identifying 13 categories of GDPR violation terms across 299 apps that provided privacy policies.

dnSpy: A .NET assembly reverse engineering tool used to extract C# source code from Mono-based Unity app DLL files (Assembly-CSharp.dll), enabling analysis of Unity IAP vulnerabilities and biometric data usage in C# code.

AI and Machine Learning Tools

Taint Analysis Framework (FlowDroid-based): A static program analysis technique adapted from FlowDroid to track data propagation paths from sensitive taint sources to risky taint sinks. Applied for both Unity IAP vulnerability detection and PII data leaks identification.

Static Native Binary Analysis Framework: Used to analyze IL2CPP-based Unity apps by processing the compiled binary libil2cpp.so together with the function mapping file global-metadata.dat, enabling identification of IAP and biometric data vulnerabilities in native Unity code.

Natural Language Processing (NLP) for Privacy Policies: NLP techniques integrated within PolicyLint are used to parse and semantically interpret privacy policy text, extracting structured entity-action-data triples and cross-referencing them against code analysis results.

7. CRITICAL ANALYSIS OF SE PRACTICES

The application of Software Engineering principles in the VR-SP Detector study is both systematic and impactful, providing a strong foundation for a reliable and reproducible security assessment framework. The modular architecture, with its five clearly delineated analysis modules, demonstrates exemplary separation of concerns. Each module can be independently validated, updated, or replaced—for instance, the privacy policy analysis module could be upgraded to support newer NLP models without affecting the taint analysis or manifest analysis components. This design flexibility is a hallmark of well-engineered software systems.

The study's commitment to testing and validation is particularly commendable. By manually verifying 20% of taint analysis results and 50 privacy policy GDPR checks, the authors demonstrate a principled approach to quality assurance that goes beyond automated tool outputs. The explicit reporting of false positive rates—and potential sources of inaccuracy in GDPRWise for non-English policies—reflects transparency that is central to sound SE practice.

The iterative evolution from a 500-app preliminary study to the current 900-app study, with the addition of TPL analysis and user review modules, exemplifies an agile, incremental development philosophy. This approach allowed the research team to validate core components before expanding scope, reducing the risk of systemic errors propagating through the full analysis pipeline.

However, several limitations in the application of SE practices are notable. First, the static analysis approach, while scalable and automatable, inherently cannot capture runtime behaviors such as dynamic code loading or network traffic-based data exfiltration. Second, the reliance on pattern matching for vulnerability detection creates a boundary where novel or obfuscated vulnerability patterns may evade detection. The paper acknowledges that code obfuscation limits the completeness of decompiled code analysis. Third, the GDPR compliance checker (GDPRWise) exhibited false positives for policies in non-English languages and PDF-formatted links, suggesting that the tool integration layer requires additional robustness for internationalized applications.

The focus on Unity-based VR apps, while justified by Unity's market dominance, limits the generalizability of the VR-platform-related vulnerability findings to apps developed with other engines such as Unreal Engine or libGDX. Despite these limitations, the VR-SP Detector represents a significant and well-engineered contribution to the field of mobile and VR application security analysis.

8. REAL LIFE CASE STUDY APPLYING THE SE PRINCIPLES

Case Study: Security Hardening of a Commercial VR Social Platform

System Description:

Consider a large-scale commercial VR social platform—analogous to VRChat or Rec Room—operating in the metaverse context. The platform hosts hundreds of thousands of active users who interact in virtual spaces, attend virtual events, and engage in social activities. The platform is built on Unity, runs on Meta Quest devices, and continuously releases updates to add new social features, virtual commerce capabilities, and enhanced avatar customization using biometric data including face-tracking and hand-tracking.

As the platform scales, its development team faces mounting security and privacy challenges. User biometric data collected for avatar animation is stored and processed without explicit privacy policy declarations. Several third-party analytics SDKs integrated into the app transmit sensitive user data to external servers without the users' knowledge. Security audits are infrequent and manually conducted, making it difficult to keep pace with the continuous release cycle. Furthermore, the app has received user reviews expressing concern about microphone access and unexpected permission requests, but these reviews have not been systematically analyzed to inform security decisions.

To address these challenges, the platform's engineering team adopts an automated security and privacy assessment pipeline modeled on the VR-SP Detector framework.

Application of SE Principles:

Modularity: The assessment pipeline is organized into independent modules mirroring the VR-SP Detector: a manifest scanner, a static vulnerability analyzer, a taint analysis engine, a privacy policy consistency checker, and a review sentiment analyzer. Each module is integrated into the platform's CI/CD pipeline as a separate stage, allowing targeted updates without disrupting the overall pipeline.

Abstraction: The complex process of APK decompilation and bytecode analysis is abstracted behind the Androguard-based scanner module, allowing security engineers to query vulnerability findings through a high-level dashboard rather than interacting with raw Smali code.

Maintainability: Pre-defined vulnerability pattern rules are stored in a versioned configuration repository, allowing the security team to add new patterns as novel VR-specific vulnerabilities are

discovered, without modifying the core analysis engine. When Meta releases new SDK permissions (e.g., new biometric tracking APIs), the manifest scanner's permission taxonomy is updated by appending entries to the configuration, not by rewriting the scanner.

Reusability: The taint analysis engine, originally developed for PII data leak detection in Android apps, is reused and extended to cover VR-specific taint sources such as OVRHand tracking methods and OVRFaceExpressions APIs, avoiding the need to build a new analysis engine from scratch.

Testing and Validation: Each release of the platform is automatically scanned by the assessment pipeline. Security findings are classified by severity, and high-severity issues—such as biometric data collection without manifest permissions—trigger automated deployment blocks in the CI/CD pipeline, ensuring that vulnerable builds do not reach production.

Scalability: As the platform expands to support new VR devices (e.g., Meta Quest 3, Apple Vision Pro), the assessment pipeline is extended by adding new device-specific permission taxonomies and SDK detection rules. The batch processing architecture ensures that even large app datasets can be analyzed within acceptable time frames.

By adopting this SE-principled assessment framework, the platform's development team achieves a 60% reduction in security review cycles, eliminates biometric permission inconsistencies discovered in pre-release audits, and produces privacy policies that pass GDPR compliance checks—demonstrating the concrete business value of applying rigorous SE practices to VR app security.

9. MAPPING SE PRINCIPLES

SE Principle	Application in VR-SP Detector	Example from Case Study	Outcome
Modularity	Five independent analysis modules (manifest, OS vuln., PII, privacy policy, review)	Separate CI/CD stages for manifest scan, taint analysis, policy check	Isolated debugging; independent updates per module
Abstraction	Androguard abstracts APK decompilation; PolicyLint abstracts NLP parsing	High-level security dashboard over raw Smali analysis	Simplified interface for security engineers
Maintainability	Pattern rules in versioned config; modular TPL detection via LibScan	New biometric API patterns added via config, not code change	Reduced technical debt; faster adaptation to new SDKs
Reusability	FlowDroid-based taint analysis reused from Android; patterns adapted from COVID-19 app study	PII taint engine extended to cover OVRHand/OVRFace APIs	Faster development; leveraged proven methodologies
Testing & Validation	20% manual validation of taint paths; 50 manual GDPR policy checks	High-severity findings trigger automated deployment blocks	High precision; trustworthy outputs
Scalability	900-app analysis (5× OVRSEEN); extensible to new VR devices and engines	Pipeline extended to Meta Quest 3 and Apple Vision Pro	Handles growing VR ecosystem without architectural rework
Lifecycle Practices	Iterative study: 500-app preliminary study expanded to 900 apps with new modules	Assessment pipeline integrated into every CI/CD release cycle	Continuous improvement; security embedded in development

Table 9.1: Mapping of SE Principles to VR-SP Detector and Case Study Applications

10. BENEFITS OF APPLYING SE PRINCIPLES TO THE SYSTEM

1. Improved Design Quality

Modularity ensures that each analysis component addresses a single, well-defined concern. In the VR-SP Detector, this manifests as five clearly delineated modules that can be independently verified and enhanced. In the e-commerce-style VR platform case study, this means that upgrading the biometric permission taxonomy does not require re-testing the taint analysis engine.

2. Enhanced Reliability

The integration of rigorous testing and validation mechanisms—including manual spot-checks of taint analysis paths and GDPR compliance verifications—ensures that the assessment outputs are trustworthy. False positive rates are monitored and minimized, increasing the confidence of developers and security engineers in the generated findings.

3. Greater Reusability and Reduced Development Effort

By reusing established analysis tools (Androguard, FlowDroid, PolicyLint) and adapting vulnerability patterns from prior studies, the VR-SP Detector avoids redundant engineering effort. This accelerates the development of VR-specific extensions while maintaining the robustness of the underlying analysis engines.

4. Increased Maintainability Over Time

The structured, rule-based vulnerability detection approach ensures that the system remains maintainable as new VR SDKs and APIs are released. Adding support for a new biometric tracking API, for instance, requires only the addition of new function signatures to the pre-defined pattern tables rather than architectural modifications.

5. Scalability for Large-Scale Deployment

The automated pipeline design enables the VR-SP Detector to scale to datasets far larger than the 900 apps analyzed in this study. The documented GitHub repository and standardized dataset ensure that other researchers and industry practitioners can deploy the tool on their own app collections, multiplying the impact of the work.

6. Transparency and Reproducibility

The documentation of pre-defined vulnerability patterns (Table II), keyword taxonomies (Table III), and TPL detection rules, combined with the public release of the tool and dataset, ensures that all findings are reproducible and verifiable. This is a critical benefit for academic peer review and industry adoption alike.

7. Policy and Regulatory Alignment

By integrating GDPR compliance checking and privacy policy consistency analysis into the assessment framework, the VR-SP Detector directly supports developers in meeting legal obligations. The eight development recommendations derived from the study provide actionable guidance that can be embedded into VR app development workflows, reducing legal and reputational risks for app developers.

11. CONCLUSION

This report presented a comprehensive analysis of the research paper 'An Empirical Study on Meta Virtual Reality Applications: Security and Privacy Perspectives,' examining its methodology, findings, and alignment with core Software Engineering principles. The study introduced the VR-SP Detector, a well-engineered and scalable framework that integrates static code analysis, privacy policy NLP analysis, and user review analysis to provide a multi-dimensional assessment of security and privacy risks in 900 Meta VR applications.

From a Software Engineering perspective, the study exemplifies many key principles. Modularity is evident in the five independently operable analysis modules. Abstraction is achieved through the integration of domain-specific analysis tools that hide underlying complexity. The iterative extension from a preliminary 500-app study reflects lifecycle practices aligned with agile software development. Rigorous validation through manual sampling demonstrates a commitment to quality assurance that is central to professional SE practice.

The empirical findings are both significant and concerning. Widespread absence of root detection, pervasive biometric data collection without permission declarations, and the near-universal non-compliance with GDPR in app privacy policies highlight a sector where security and privacy engineering practices lag far behind the pace of technological innovation. The study's eight development recommendations—covering manifest security flags, root detection, encryption standards, TPL management, biometric permission compliance, and privacy policy transparency—provide a concrete roadmap for improving VR app security.

The real-world case study of a commercial VR social platform demonstrated how the SE principles embedded in the VR-SP Detector can be directly applied in industry settings, yielding measurable improvements in security posture, development efficiency, and regulatory compliance. As VR and metaverse applications continue to proliferate and collect increasingly sensitive user data, frameworks such as the VR-SP Detector will become indispensable tools for ensuring that innovation is matched by responsibility. Future work should extend the framework to support dynamic analysis, cover additional VR platforms (HTC VIVE, Sony PlayStation VR 2), and analyze apps developed with alternative engines such as Unreal Engine, further broadening the scope and impact of VR security research.

12. REFERENCES

- [1] H. Guo, H.-N. Dai, X. Luo, G. Xu, F. He, and Z. Zheng, "An Empirical Study on Meta Virtual Reality Applications: Security and Privacy Perspectives," *IEEE Transactions on Software Engineering*, vol. 51, no. 5, pp. 1437–1454, May 2025.
- [2] R. Trimananda, H. Le, H. Cui, J. T. Ho, A. Shuba, and A. Markopoulou, "OVRseen: Auditing network traffic and privacy policies in Oculus VR," in *Proc. 31st USENIX Security Symposium*, Boston, MA, USA, Aug. 2022, pp. 3789–3806.
- [3] C. Zuo and Z. Lin, "Playing without paying: Detecting vulnerable payment verification in native binaries of Unity mobile games," in *Proc. 31st USENIX Security Symposium*, Boston, MA, USA, Aug. 2022, pp. 3093–3110.
- [4] S. Arzt et al., "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *Proc. 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation*, New York, NY, USA, 2014, pp. 259–269.
- [5] B. Andow et al., "PolicyLint: Investigating internal privacy policy contradictions on Google Play," in *Proc. 28th USENIX Security Symposium*, Santa Clara, CA, Aug. 2019, pp. 585–602.
- [6] Y. Wu, C. Sun, D. Zeng, G. Tan, S. Ma, and P. Wang, "LibScan: Towards more precise third-party library identification for Android applications," in *Proc. 32nd USENIX Security Symposium*, Anaheim, CA, Aug. 2023, pp. 3385–3402.
- [7] P. Casey, I. Baggili, and A. Yarramreddy, "Immersive virtual reality attacks and the human joystick," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 550–562, Mar./Apr. 2021.
- [8] H. Guo, H.-N. Dai, X. Luo, Z. Zheng, G. Xu, and F. He, "An empirical study on Oculus virtual reality applications: Security and privacy perspectives," in *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*, New York, NY, USA: ACM, 2024.
- [9] Y. Huang, Y. J. Li, and Z. Cai, "Security and privacy in metaverse: A comprehensive survey," *Big Data Mining and Analytics*, vol. 6, no. 2, pp. 234–247, 2023.
- [10] H. Li, L. Yu, and W. He, "The impact of GDPR on global technology development," *Journal of Global Information Technology Management*, vol. 22, no. 1, pp. 1–6, 2019.

-
- [11] Fortune Business Insights, "Virtual reality market size, share and COVID-19 impact analysis, 2023-2030." [Online]. Available: <https://www.fortunebusinessinsights.com/industry-reports/virtual-reality-market-101378>
- [12] D. C. Nguyen, E. Derr, M. Backes, and S. Bugiel, "Short text, large effect: Measuring the impact of user reviews on Android app security & privacy," in Proc. IEEE Symposium on Security and Privacy (SP), 2019, pp. 555–569.
- [13] R. Sun et al., "An empirical assessment of global COVID-19 contact tracing applications," in Proc. IEEE/ACM 43rd Int. Conf. Software Engineering (ICSE), 2021, pp. 1085–1097.